Aprendizagem 2023/24

Homework II – Group 024

(ist105887, ist106228)

I. Pen-and-paper

1)

$$\left(\begin{pmatrix}1&1\\1&3\\1&6\\1&9\\1&8\end{pmatrix}^T\cdot\begin{pmatrix}1&1\\1&3\\1&6\\1&9\\1&8\end{pmatrix}\right)^{(-1)}\cdot\begin{pmatrix}1&1\\1&3\\1&6\\1&9\\1&8\end{pmatrix}^T\cdot\begin{pmatrix}1{,}25\\7\\2{,}7\\3{,}2\\5{,}5\end{pmatrix}=\begin{pmatrix}\frac{3747}{1130}\\\frac{257}{2260}\end{pmatrix}$$

$\check{Z} = \frac{3747}{1130} - \frac{257}{2260}\,\Phi(y1,y2)$

2)

$$\left(\begin{pmatrix}1&1\\1&3\\1&6\\1&9\\1&8\end{pmatrix}^T\cdot\begin{pmatrix}1&1\\1&3\\1&6\\1&9\\1&8\end{pmatrix}+\begin{pmatrix}1&0\\0&1\end{pmatrix}\right)^{(-1)}\cdot\begin{pmatrix}1&1\\1&3\\1&6\\1&9\\1&8\end{pmatrix}^T\cdot\begin{pmatrix}1{,}25\\7\\2{,}7\\3{,}2\\5{,}5\end{pmatrix}=\begin{pmatrix}\frac{1709}{940}\\\frac{913}{2820}\end{pmatrix}$$

$\check{Z} = \frac{1709}{940} - \frac{913}{2820}\,\Phi(y1,y2)$

The coefficients for the Ridge regression are smaller, since the penalty factor stabilizes and smoothes the model, making it more robust to outliers and less sensitive to the training data, improving its generalization.

3)

x6:

No penalty: $\frac{3747}{1130} - \frac{257}{2260} * 4 = 3.77080$

With penalty: $\frac{1709}{940} - \frac{913}{2820} * 4 = 3.11312$

x7:

No penalty: $\frac{3747}{1130} - \frac{257}{2260} * 2 = 3.54336$

With penalty: $\frac{1709}{940} - \frac{913}{2820} * 2 = 2.46560$

X8:

No penalty: $\frac{3747}{1130} - \frac{257}{2260} * 5 = 3.88451$

With penalty: $\frac{1709}{940} - \frac{913}{2820} * 5 = 3.43688$

RMSE for non-regularized linear regression:

$$\sqrt{\frac{(3.77080-0.7)^2+(3.54336-1.1)^2+(3.88451-2.2)\text{^}2}{3}}=2.46559$$

RMSE for Ridge regression:

$$\sqrt{\frac{(3.11312-0.7)^2+(2.46560-1.1)^2+(3.43688-2.2)\text{^}2}{3}}=1.28122$$

The non-regularized linear regression performs poorly in comparison to the Ridge regression as expected. This can occur because the linear regression without being regularized is very propense to overfitting, as mentioned before.

4)

$$\begin{pmatrix} 0{,}1 & 0{,}1 \\ 0{,}1 & 0{,}2 \\ 0{,}2 & 0{,}1 \end{pmatrix}\cdot\begin{pmatrix} 1 \\ 1 \end{pmatrix}+\begin{pmatrix} 0{,}1 \\ 0 \\ 0{,}1 \end{pmatrix}=\begin{pmatrix} \frac{3}{10} \\ \frac{3}{10} \\ \frac{2}{5} \end{pmatrix}$$

$$\begin{pmatrix} 1 & 2 & 2 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{pmatrix}\cdot\begin{pmatrix} \frac{3}{10} \\ \frac{3}{10} \\ \frac{2}{5} \end{pmatrix}+\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}=\begin{pmatrix} \frac{27}{10} \\ \frac{23}{10} \\ 2 \end{pmatrix}$$

Softmax(2.7) = e^2.7/(e^2.7+e^2.3+e^2) = 0.4615
Softmax(2.3) = e^2.3/(e^2.7+e^2.3+e^2) = 0.3093
Softmax(2) = e^2/(e^2.7+e^2.3+e^2) = 0.2292

A is the predicted output.
t=(1,0,0)
Stochastic Gradient descent weight update rule:

$$w_{l,j}=w_{l,j}-\eta\cdot\frac{\partial L}{\partial w_{l,j}}$$

Using the chain rule we have, for each class l and for each weight w_j:

$$\frac{\partial L}{\partial w_{l,j}}=\frac{\partial L}{\partial X[out]_l}\cdot\frac{\partial X[out]_l}{\partial z_l}\cdot\frac{\partial z_l}{\partial w_{l,j}}$$

$\frac{\partial L}{\partial X[out]}$ is the derivative of the cross-entropy loss with respect to the softmax output, which is trivial:

$$\frac{\partial L}{\partial X[out]}=-\frac{t}{X[out]}$$

To calculate $\frac{\partial X[out]_l}{\partial z_l}$, we need to consider two cases:

$\frac{\partial X[out]_l}{\partial z_l} = X[out]_l(1\text{-}X[out]_l)$, for the correct class $l = c$

$\frac{\partial X[out]_k}{\partial z_l} = -X[out]_k X[out]_l$, for the other classes

Putting all together we get:

$\frac{\partial L}{\partial z_l} = X[out]_l - 1$, for the correct class $l = c$

$\frac{\partial L}{\partial z_l} = X[out]_l$, for the other classes

So attending to the cross-entropy loss characteristics, we can generalize to:

$\frac{\partial L}{\partial z_l} = X[out]_l - t_l$

Last but not least, we need to calculate $\frac{\partial z_l}{\partial w_{l,j}}$. We know that z_l is a linear combination of weights and inputs, so if we calculate the derivative of z_l in respect to a certain $w_{l,j}$, we will get the input feature x_j.

Concluding, we get:

$\frac{\partial L}{\partial w_{l,j}} = x_j\ (X[out]_l - t_l)$

Now we have everything necessary to start updating the weights with the formula:

$w_{l,j} = w_{l,j} - \eta \cdot x_j \cdot (X[out]_l - t_l)$

For simplification purposes we will use matrix multiplications, so we get:

$W = W - \eta \cdot X^T \cdot (X^{[out]} - T)$

Where W is the weight matrix of the layer, X^T is the transpose of the input matrix for the layer that its being calculated, $X^{[out]}$ is the predicted probabilities matrix and T is the matrix of true labels.

For the second layer:

Calculation for the weights matrix:

$$\begin{pmatrix} 1 & 2 & 2 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{pmatrix} - 0{,}1 \cdot \left(\left(\begin{pmatrix} 0{,}4615 \\ 0{,}3093 \\ 0{,}2292 \end{pmatrix} - \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \right) \cdot \begin{pmatrix} \frac{3}{10} \\ \frac{3}{10} \\ \frac{2}{5} \end{pmatrix}^T \right) = \begin{pmatrix} 0{,}986 & 1{,}986 & 1{,}982 \\ 1{,}021 & 2{,}021 & 1{,}028 \\ 0{,}993 & 0{,}993 & 0{,}991 \end{pmatrix}$$

Calculation for the bias matrix:

$$\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} - 0{,}1 \cdot \left(\begin{pmatrix} 0{,}4615 \\ 0{,}3093 \\ 0{,}2292 \end{pmatrix} - \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \right) = \begin{pmatrix} 0{,}954 \\ 1{,}069 \\ 0{,}977 \end{pmatrix}$$

For the first layer:
First calculate $\frac{\partial L}{\partial w_1}$

$$\begin{pmatrix} 0{,}98615 & 1{,}98615 & 1{,}98154 \\ 1{,}02072 & 2{,}02072 & 1{,}02763 \\ 0{,}99312 & 0{,}99312 & 0{,}99083 \end{pmatrix}^T \cdot \left(\begin{pmatrix} 0{,}4615 \\ 0{,}3093 \\ 0{,}2292 \end{pmatrix} - \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \right) \cdot \begin{pmatrix} 1 & 1 \end{pmatrix} = \begin{pmatrix} -0{,}022 & -0{,}022 \\ -0{,}251 & -0{,}251 \\ 0{,}432 & 0{,}432 \end{pmatrix}$$

Then update the weights:

$$\begin{pmatrix} 0{,}1 & 0{,}1 \\ 0{,}1 & 0{,}2 \\ 0{,}2 & 0{,}1 \end{pmatrix} - 0{,}1 \cdot \begin{pmatrix} \frac{-891199}{40000000} & \frac{-891199}{40000000} \\ \frac{-10059199}{40000000} & \frac{-10059199}{40000000} \\ \frac{86358981}{200000000} & \frac{86358981}{200000000} \end{pmatrix} = \begin{pmatrix} 0{,}102 & 0{,}102 \\ 0{,}125 & 0{,}225 \\ 0{,}157 & 0{,}057 \end{pmatrix}$$

And the biases:

$$\begin{pmatrix} 0{,}1 \\ 0 \\ 0{,}1 \end{pmatrix} - 0{,}1 \cdot \begin{pmatrix} -0{,}00218 \\ -0{,}02518 \\ 0{,}4322 \end{pmatrix} = \begin{pmatrix} 0{,}100 \\ 0{,}003 \\ 0{,}057 \end{pmatrix}$$

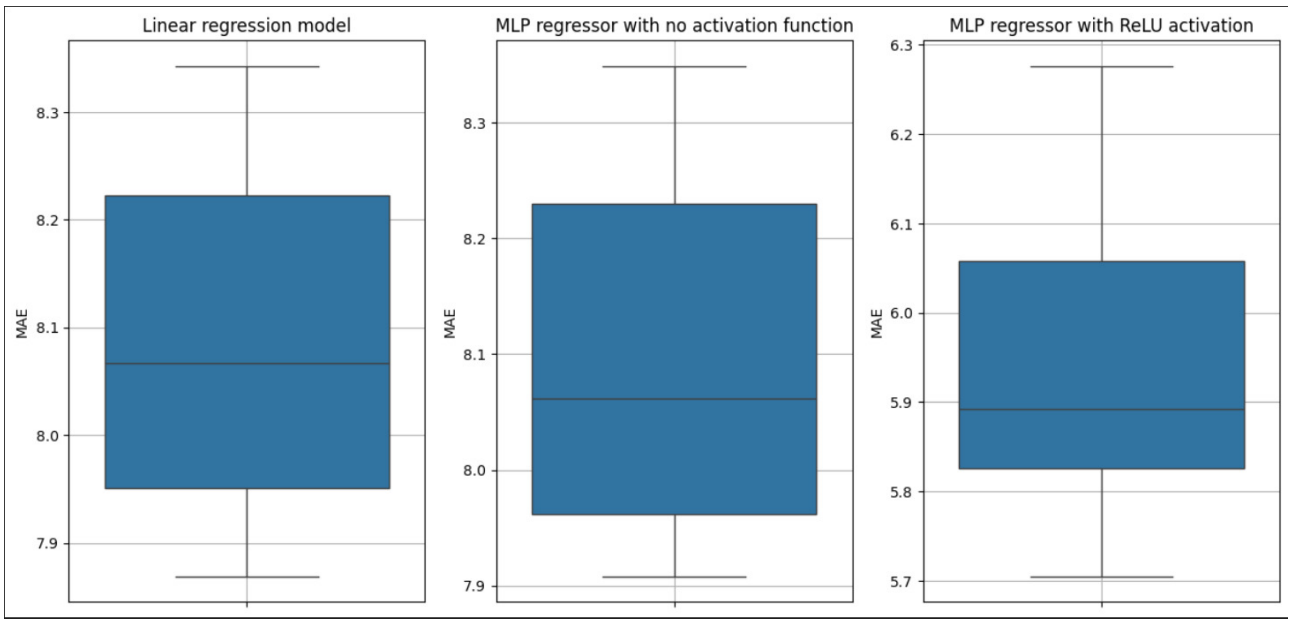
Aprendizagem 2023/24

Homework II – Group 024

(ist105887, ist106228)

II. Programming and critical analysis

5)



6)

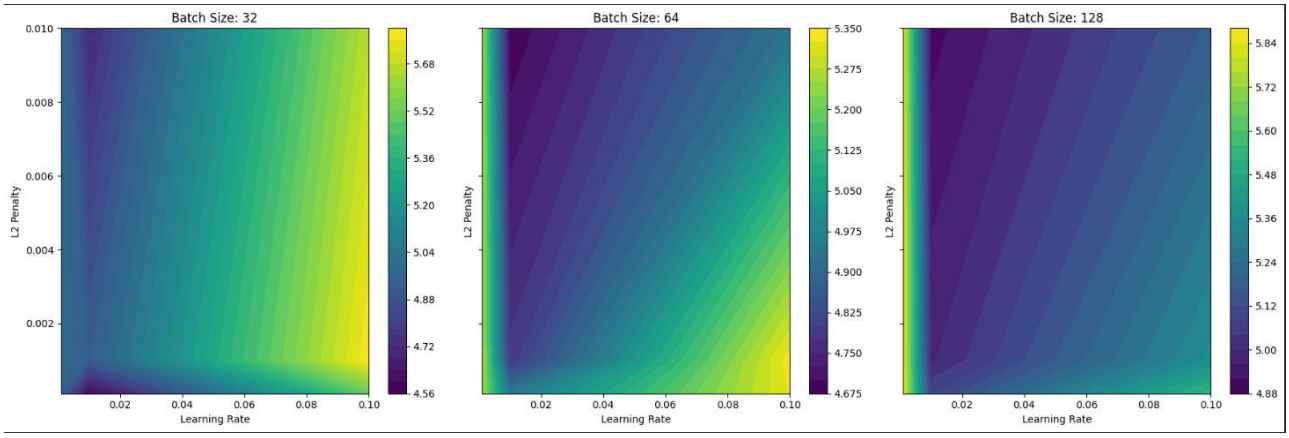
O modelo de regressão linear e o MLP sem funções de ativação apresentam desempenho semelhante, com valores de MAE bastante perto dos do outro. Isto deve-se ao facto de um MLP sem funções de ativação ser semelhante a um modelo de regressão linear, independentemente do número de camadas, pois o output das layers é essencialmente a soma linear dos inputs. Sem transformações não lineares, a rede não é capaz de modelar relações complexas nos dados, o que limita seu desempenho.

Por contraste, o MLP com ativação ReLU apresenta valores de MAE significativamente mais baixos. função de ativação ReLU introduz a não linearidade no modelo, permitindo que o MLP aprenda padrões mais complexos nos dados que os modelos lineares não capazes de captar. Isso resulta em melhor desempenho, logo, menor MAE. Assim, é evidente a importância das funções de ativação nas redes neuronais.

7)

Ao ajustar os hyperparameters para um MLPRegressor, os trade-offs envolvendo a regularização L2 (alfa), a taxa de aprendizagem (learning_rate_init) e o tamanho do batch são evidenciadas. Um alfa pequeno permite ao modelo capturar padrões complexos mas corre o risco de overfitting, enquanto um alfa grande reduz o overfitting à custa de um potencial underfitting. A taxa de aprendizagem afeta a rapidez com que o modelo actualiza os pesos, logo, uma taxa baixa assegura a estabilidade mas atrasa a convergência, enquanto uma taxa elevada acelera o treino mas pode perturbá-lo.

O tamanho do batch influencia a qualidade da atualização do gradiente, assim, tamanhos mais pequenos podem melhorar a generalização através de atualizações ruidosas mas podem tornar o treino mais lento, enquanto tamanhos maiores estabilizam as atualizações com o risco de redução da generalização.


Batch Size: 32
Batch Size: 64
Batch Size: 128
L2 Penalty
Learning Rate
0.010
0.008
0.006
0.004
0.002
0.02
0.04
0.06
0.08
0.10
5.68
5.52
5.36
5.20
5.04
4.88
4.72
4.56
5.350
5.275
5.200
5.125
5.050
4.975
4.900
4.825
4.750
4.675
5.84
5.72
5.60
5.48
5.36
5.24
5.12
5.00
4.88

END